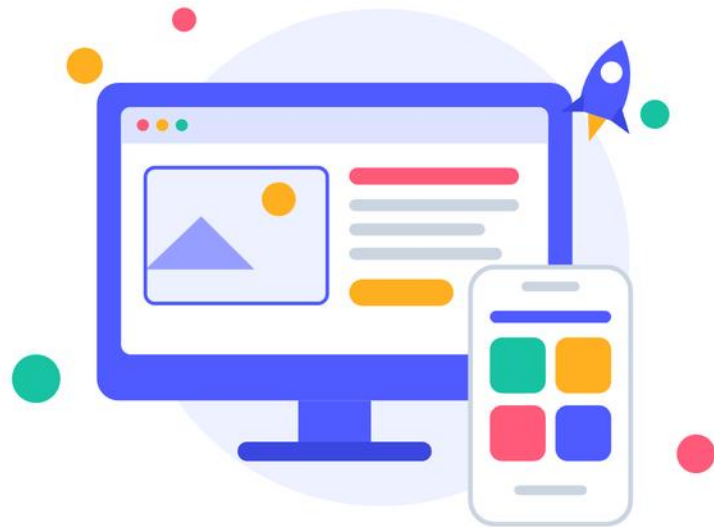


WEEK 04 · FLOW · POSITION · FLEXBOX

# WHERE THE BOXES GO

Last week you learned to style a box. Today you learn to put one somewhere.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN



## WHERE WE ARE

Week 4 of 12. You have a page with a design language and no layout.

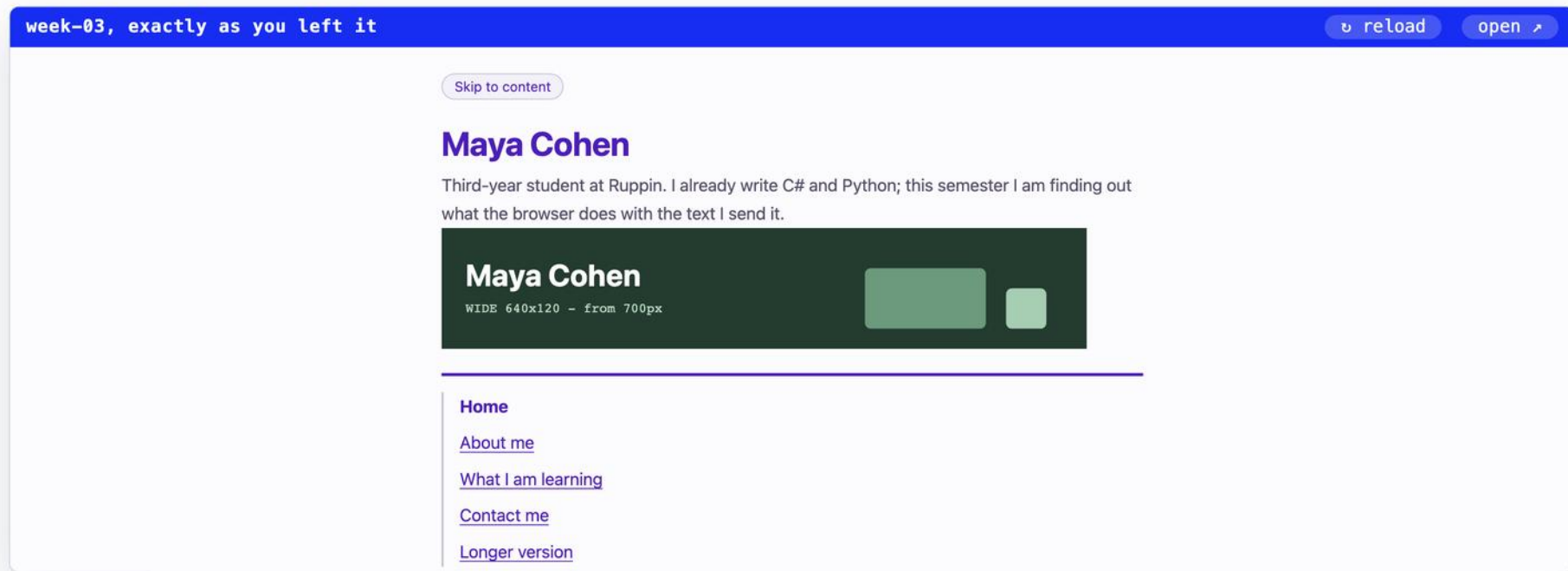
## THE NEXT 180 MINUTES

Five cycles, you typing in every one: a navbar · a badge and a sticky header · a card row that wraps · one prompt to Claude · a menu that closes on a phone. Break at 90. The assignment at 145.

## WHAT THIS UNLOCKS

Week 5 is Grid, and the decision rule "Grid for the page, Flex inside the component" only means something once you know what Flex is.

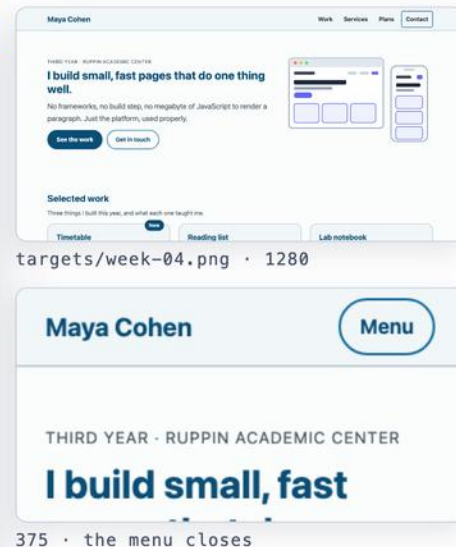
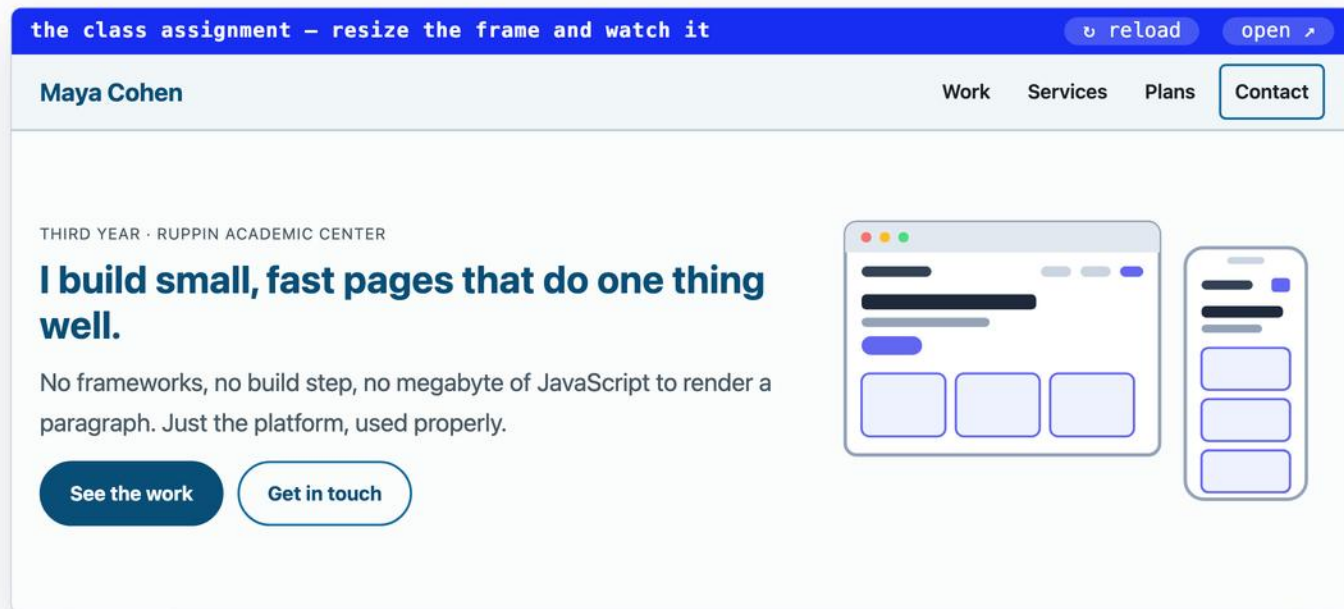
# THIS IS EVERYTHING YOU CAN BUILD RIGHT NOW



► **SAY IT OUT LOUD** — this page is **GOOD**. And it is a **SINGLE COLUMN**, because that is the only shape you know how to make.

TARGET · MINUTE 0

# THIS IS WHERE YOU WILL BE AT MINUTE 180



► **THE POINT** — everything that moves when you resize this does so without a single line that mentions a screen size. You type every piece of it today: once in a cycle, once in the assignment.

## AI THIS WEEK

# TUTOR. IT EXPLAINS; YOU TYPE.

LAYER · WEEKS 1-5 · PRIVATE TUTOR

## ALLOWED

Ask it to explain the two axes, a stacking context, `flex-basis`. Ask it to criticise CSS *you* typed. Ask "why doesn't my rule apply?" — then check the answer in DevTools, not in your gut.

## NOT ALLOWED

Pasting generated CSS into the assignment. **You type every character.** What to check this week: it copies the dialect of the CSS you paste and reaches for old habits. You get the answer that works at the width you looked at — so drag the width slowly from 1600 to 320 and test the one you did not.

## NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Cycle 4 is one prompt, twice, in whatever chat you have.

CYCLE 1 · 25 MIN

# FROM A STACK TO A NAVBAR

A link, then a list with bullets — exactly where normal flow put them. A handful of declarations later it is a navbar, and not one of them is a width.

## 1 I DO

10 MIN

3 on the next two slides, 7 in the editor · live-code/01-stack-to-row · from an empty bar to brand-left, links-right

## 2 WE DO

6 MIN

you type it on your copy · compare with checkpoint 1

## 3 YOU DO

7 MIN

warmup/ — the gold bar, five declarations, and the push that needs two

## 4 DEBRIEF

2 MIN

flex on the item · the ul padding · ↗ reference

YOU DO · AI-FREE

## CYCLE 1 · BEFORE YOU TYPE

# NOTHING YOU HAVE WRITTEN SO FAR HAS PLACED ANYTHING

Every page you have made was laid out by a rule you were never told. It is **two sentences long**.

## A BLOCK BOX

Takes the full width offered; the next one starts underneath.

```
p h1 div section li
```

## AN INLINE BOX

Takes the width of its content; the next one continues on the line.

```
a span strong em img
```

CYCLE 1 · BEFORE YOU TYPE

# `display` DECIDES WHAT KIND OF BOX

It is not decoration and it is not visibility. It answers one question: *what sort of box does this element generate?*

## `block`

Full width offered · stacks · width, height and all margins work.

## `inline`

Content width · flows in a line · width, height and vertical margin ignored.

## `inline-block`

Content flow, block sizing. Both, and one famous problem.

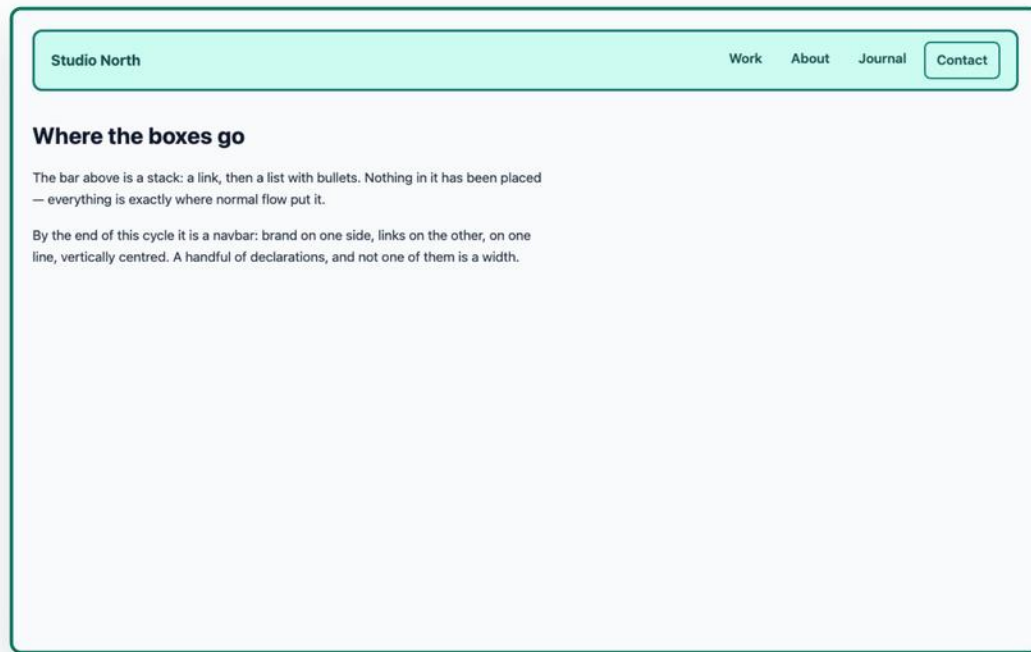
## `none`

No box at all. Not "invisible" — absent.



## CHECKPOINT 1

# YOUR SCREEN SHOULD LOOK LIKE THIS



**Compare** — `live-code/01-stack-to-row/checkpoint.png` is this picture at 1280; `live-code/01-stack-to-row/end/` is the code behind it. Brand left, links pushed right, one line, Contact outlined.

CYCLE 2 · 25 MIN

# THE BADGE, THE ANCHOR, AND THE HEADER THAT STAYS

Three uses of `position`, and not one of them places a column: a badge on a corner, the box it is measured from, a header that stays — above the content.

## 1 I DO

9 MIN

2 on the next slide, 7 in the editor ·  
live-code/02-badge-and-anchor ·  
the badge flies, the anchor, sticky,  
the 999 that loses

## 2 WE DO

6 MIN

type it · checkpoint 2 · `offsetParent`  
in the console

## 3 YOU DO

8 MIN

a second badge, bottom-left of the  
third tile · verify with `offsetParent`

## 4 DEBRIEF

2 MIN

the diagnostic sentence · ↗  
reference

YOU DO · AI AS TUTOR

# EVERY BOX IS MEASURED AGAINST ANOTHER BOX

**containing block** The box your box's sizes and offsets are resolved against.

**in normal flow** Your nearest block-level ancestor's **content box**.

**why you care** `width: 50%` is 50% *of that*. So is `left: 10%`.

Change which box that is, and every percentage in the element changes meaning without you touching it. **Cycle 2 is entirely about changing it.**

```
1 .parent {  
2   width: 600px;    /* the containing block */  
3 }  
4  
5 .child {  
6   width: 50%;      /* 300px */  
7 }
```

CYCLE 2 · BEFORE YOU TYPE

# TAKING A BOX OUT OF THE CONVERSATION

Normal flow is a negotiation between siblings. `position` is how one box *leaves* it — and the cost is that nothing reserves its space any more.

**static**

The default. In flow. Offsets ignored.

**relative**

In flow, keeps its space, moves visually.

**Anchors its children.**

**absolute**

Out of flow. Measured from the nearest positioned ancestor.

**fixed**

Out of flow. Measured from the viewport. Does not scroll.

**sticky**

In flow, then pins at a threshold. The one you will ship.



# ALL FIVE, ON ONE SCROLLING PAGE

[demos/03-position/02-five-values](#)[↶ reload](#)[open ↷](#)

[static](#)[relative](#)[absolute](#)[fixed](#)[sticky](#)

THE DASHED FRAME IS THE CONTAINER. IT IS `position: relative`.

Text above the box.

where it would be in normal flow

the box

`position: static`

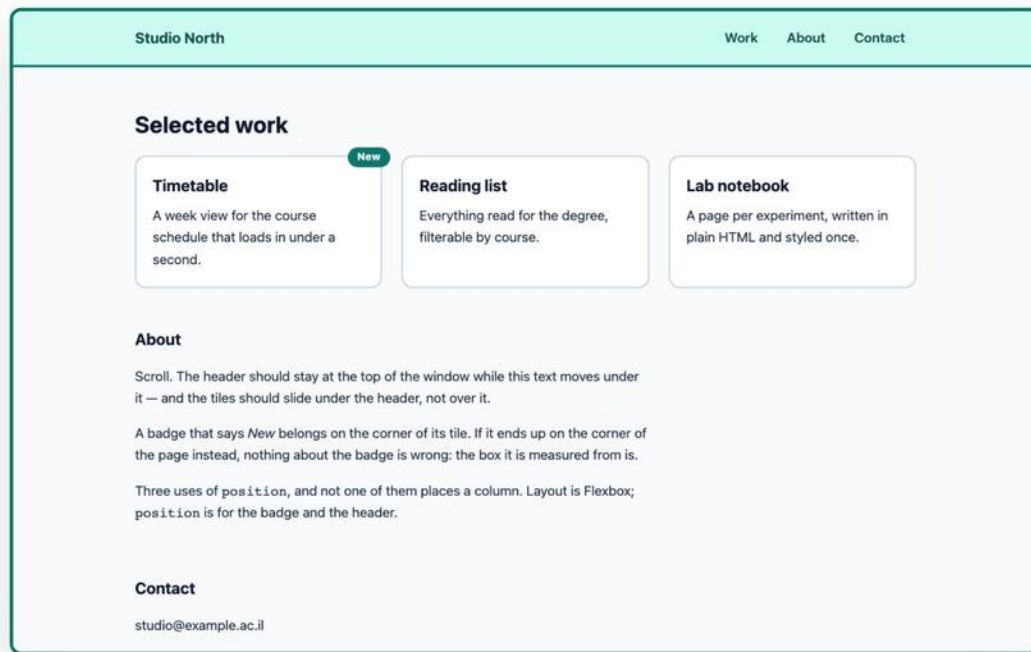
|                       |                                             |
|-----------------------|---------------------------------------------|
| still in normal flow? | yes – this IS normal flow                   |
| offsets measured from | nothing, top and left are ignored entirely. |
| when you scroll       | scrolls with the page                       |
| container height now  | 0px                                         |
| computed position     | static                                      |

The default. Every element on every page you have written so far.  
Note that top/left do nothing here – offsets need a position first.

► **HANDS-ON** [demos/03-position/02-five-values/](#) — Start on `relative` and point at the dashed ghost: **THAT SPACE IS STILL RESERVED.**

## CHECKPOINT 2

## YOUR SCREEN SHOULD LOOK LIKE THIS



**Compare** — `live-code/02-badge-and-anchor/checkpoint.png` is this picture at 1280; `live-code/02-badge-and-anchor/end/` is the code behind it. The badge on the corner of the first tile, not of the page; the header stays when you scroll and the tiles slide under it.

# KEEP THIS ONE. IT IS A DIAGNOSTIC, NOT A DEFINITION.

*If it aligns to the page instead of the box you expected, you probably forgot `position: relative` on the ancestor.*

FROM THIS COURSE, BEFORE YOU WERE IN IT

## AND ITS TWO COMPANIONS

**Absolute elements won't stretch the parent's height**  
— parents may collapse.

**fixed inside a transformed ancestor** behaves like  
`absolute`. Avoid transforming high-level wrappers.

# A STACKING CONTEXT IS A SEALED BOX

When an element creates one, its children's `z-index` values are only ever compared **with each other**. Against anything outside, what competes is *the parent's own* `z-index`.

So `999` was never being compared with `1`. It was being compared with its siblings, inside a box whose own `z-index` is `auto` — and `auto` loses to `1`.

**Raising the number cannot help. It is the wrong argument.**

**position**

...that is not `static`, plus any `z-index`

**opacity**

anything below `1`

**transform**

anything other than `none`

**filter**

anything other than `none`

**will-change**

naming any of the above

**isolation**

`isolate` — the only one that is deliberate

CYCLE 3 · 30 MIN

# THE CARD ROW THAT WRAPS BY ITSELF

Three across when there is room, one under the other when there is not, nothing broken in between — and not one line that mentions a screen.

**1 I DO**

10 MIN

3 on the next three slides, 7 in the editor · live-code/03-card-row-wraps · width fails, flex-basis wins, min-width: 0, the link at the bottom

**2 WE DO**

8 MIN

type it · checkpoint 3 · a fourth card with no CSS change

**3 YOU DO**

10 MIN

a pricing row: three tiers, buttons on one line at the bottom

**4 DEBRIEF**

2 MIN

width vs flex-basis · min-width: auto · ↗ reference

YOU DO · AI-FREE

CYCLE 3 · BEFORE YOU TYPE

## ONE DIMENSION, DONE PROPERLY

*It's designed for components — navbars, cards, toolbars, forms — rather than full page grids.*

FROM THIS COURSE, BEFORE YOU WERE IN IT

**A **row** or a **column**, with real control over how the space between and around the items is distributed. That is the entire scope, and it is why it is easy.**



# TWO AXES, AND THEY MOVE

**main axis** The direction items are laid out. Set by `flex-direction`.

**cross axis** Perpendicular to it. Always.

**justify-content** Distributes along the **main** axis.

**align-items** Aligns along the **cross** axis.

Neither of those two is "horizontal" or "vertical". They are **main** and **cross**, and which is which is a decision you made one line earlier.

Where the axes point

| FLEX-DIRECTION              | MAIN         | CROSS        |
|-----------------------------|--------------|--------------|
| <code>row</code>            | horizontal → | vertical ↓   |
| <code>row-reverse</code>    | horizontal ← | vertical ↓   |
| <code>column</code>         | vertical ↓   | horizontal → |
| <code>column-reverse</code> | vertical ↑   | horizontal → |

# CONTAINER PROPERTIES · ITEM PROPERTIES

## ON THE CONTAINER

`flex-direction` · `flex-wrap` · `justify-content` ·  
`align-items` · `align-content` · `gap`

Controls the flow and the alignment of all the items at once.

## ON AN ITEM

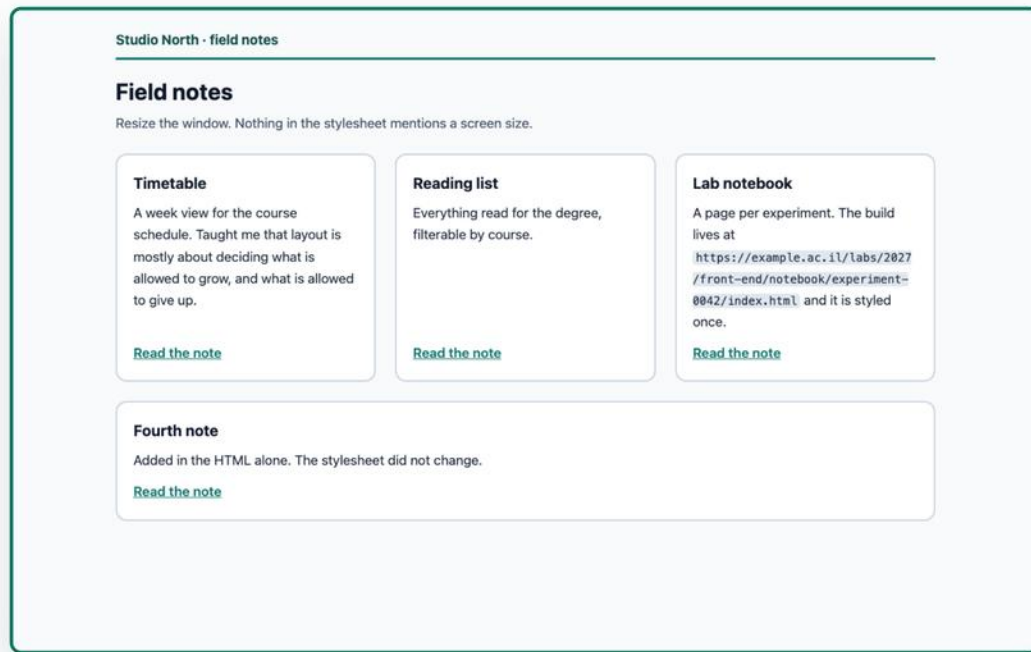
`flex-grow` · `flex-shrink` · `flex-basis` (and the  
`flex` shorthand) · `align-self` · `order`

Controls the size and the alignment of one item.

**Start with `flex-basis` to set the natural size,**  
**then tweak `flex-grow` and `flex-shrink`.**

## CHECKPOINT 3

## YOUR SCREEN SHOULD LOOK LIKE THIS



**Compare** — `live-code/03-card-row-wraps/checkpoint.png` is this picture at 1280; `live-code/03-card-row-wraps/end/` is the code behind it. Three across, equal height, links on one line, the URL broken inside its card, the fourth card alone and full width below.

BREAK

# 10 MINUTES

Back at 100. Cycle 4 is the AI cycle — have a chat window open, free or paid.

CYCLE 4 · 25 MIN

## ONE PROMPT, TWICE: "PUT THESE THREE CARDS IN A ROW"

Paste a stylesheet that has one float in it and ask for a row. What comes back is 2010 — correct at 1280, wrong at the width you did not check. Then ask again with constraints.

### 1 I DO

7 MIN

2 on the next slide, 5 in the chat ·  
ai-lab/04-floats-or-flex · P1 live · run  
it at 1280, then at 375 ·  
transcript.md if the room got  
something else

### 2 WE DO

6 MIN

everyone pastes P1 · find what is  
wrong before it is named

### 3 YOU DO

10 MIN

write P2 yourself, mobile-first, at  
most one query · run it at three  
widths, then drag 1600 to 320 · a  
journal line: claim · checked ·  
verdict

### 4 DEBRIEF

2 MIN

the claim that has to be checked  
between X and Y · ↗ exemplars

YOU DO · AI AS TUTOR

CYCLE 4 · BEFORE YOU PROMPT

# TWO MINUTES ON `float`, BECAUSE YOU WILL MEET IT

demos/07-archaeology — five ways, 1998 to 2017

↶ reload

open ↷

1 · tables (1998)

2 · inline-block (2007)

3 · absolute (2005)

4 · float (2010)

5 · flex (2017)

THE SAME THREE COLUMNS. THE MARKUP NEVER CHANGES.

Column one, with a sentence in it so the boxes have something to be.

Column two.

Column three, which is the longest of the three by some way.

A paragraph after the columns.

```
.cols { display: table; width: 100%; border-spacing: 5px }  
.col { display: table-cell; width: 33.33% }
```

What it cost: the layout lives in the markup. A screen reader announces a table with rows and cells to someone who is trying to read an article, and the page cannot be rearranged for a narrow screen at all – a table is a table at every width.

Tables are for tabular data. That has never changed.

► **HANDS-ON** demos/07-archaeology/ — Click through in order and read the "what it cost" line. Then stop.

## CHECKPOINT 4

## THREE QUESTIONS BEFORE THE NEXT CYCLE

1. Did you open the answer to P1 **at 375 and at 1280** and drag the width slowly from 1600 to 320 — not just stop at three widths?
2. Does your P2 contain at least two conditions a browser can check — "one column at 375", "at most one media query" — and does the answer meet both when you check?
3. Does your journal line contain a **width or a measurement** — a number, a tool, what you typed in the console — and not just "worked"?

**Compare** — `ai-lab/04-floats-or-flex/checkpoint-he.md` has the same three questions; `artifact/claude-p1/` and `artifact/claude-p2/` are what came back, ready to open.

CYCLE 5 · 20 MIN

# THE MENU THAT CLOSES ON A PHONE

Everything else on the page adapts with no media query. This is the one question Flexbox cannot answer — so this is the one query you are allowed.

## 1 I DO

7 MIN

2 on the next slide, 5 in the editor ·  
live-code/05-menu-closes-on-  
mobile · the rules outside the  
query, then inside · the two given  
lines · the overlay

## 2 WE DO

5 MIN

type it · checkpoint 5 · check both  
widths

## 3 YOU DO

6 MIN

Contact first in the open panel with  
order · dividers · both undone  
above 48em

## 4 DEBRIEF

2 MIN

how many queries are in your file? ·  
[↗ reference](#)

YOU DO · AI AS TUTOR

# RESPONSIVE, WITH NOTHING THAT MENTIONS A SCREEN — AND THE ONE THING THAT DOES

## THE WHOLE CARD ROW

```
1 .card-row {  
2   display: flex;  
3   flex-wrap: wrap;  
4   gap: 1.5rem;  
5 }  
6  
7 .card {  
8   flex: 1 1 17rem; /* I want 17rem.  
9                     If you cannot,  
10                    take a line. */  
11 }
```

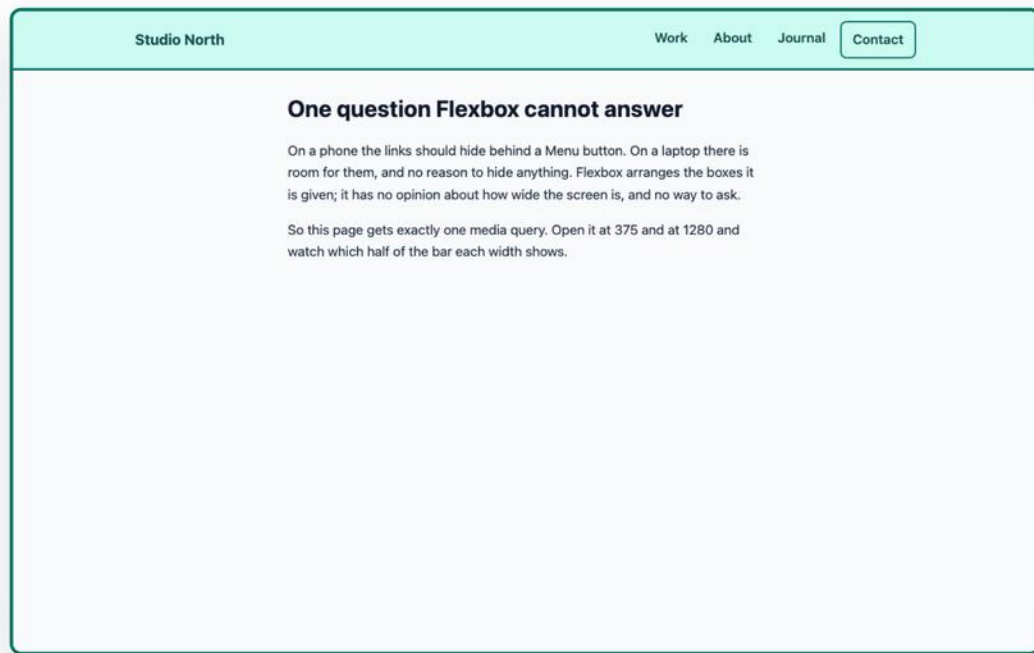
Three across at 1280. Two at 900. One at 375. **No media query, no breakpoint, no arithmetic that has to add up to 100 by hand.**

`flex-wrap` says items may take a new line rather than overflow; `flex-basis` says how much room each one wants before it gives up.

**The one thing they cannot do:** hide a Menu button on a laptop and show it on a phone. That is a question about the *viewport*, and it gets the one `@media (min-width: 48em)` in the whole file.

## CHECKPOINT 5

# YOUR SCREEN SHOULD LOOK LIKE THIS



**Compare** — `live-code/05-menu-closes-on-mobile/checkpoint.png` is this picture at 1280; `live-code/05-menu-closes-on-mobile/end/` is the code behind it. At 1280: links in a row, no button. Now 375 yourself: a Menu button, and the panel floats over the heading when opened — the heading does not move.

ASSIGNMENT · 145–175

# THE ASSIGNMENT: ASSEMBLE WHAT YOU BUILT, THEN EXTEND IT

## ASSEMBLE · CORE

`week-04/` is already in your repo from `pre-class/`. The navbar is cycle 1 + cycle 5. The badge and the sticky header are cycle 2. The card row is cycle 3. The hero is cycle 3 with two items instead of three.

## EXTEND · STRETCH AND CHALLENGE

The pricing row is cycle 3's "you do" on the assignment's markup. The badge, the sticky header and the footer row are one declaration each — you typed all of them today.

## THE RULES, AND THE EXAM

**Flexbox only. No Grid, no float, one media query.** AI as tutor — it explains, you type. Submit the SHA in Moodle. Week 13 asks you about this code, offline.

DEBRIEF · 175–180

## THREE QUESTIONS BEFORE YOU LEAVE

1. A badge with `position: absolute; top: 0; right: 0` lands in the corner of the page instead of the corner of its card. What is missing, and on which element?
2. A dropdown has `z-index: 999`. A header has `z-index: 1`. The dropdown is behind. Give me one reason — and tell me why raising 999 will not help.
3. `flex-direction: column`. Which property now centres the items left-to-right, and which one distributes them top-to-bottom?

► **GO DEEPER** — the reference deck is optional depth, and it is not examined: every value of `position`, everything that creates a stacking context, every Flexbox property in two tables, and the demos behind today's cycles. The next slide lists it.

# GO DEEPER

The reference deck for this week — every slide the lecture pointed at, in order.

[1 · The layout catalogue](#) ← spine 29

[2 · display — what kind of box](#) ← spine 5

[8 · Four ways to hide, and what each one costs](#) ← spine 25

[10 · position — the five values](#) ← spine 9

[14 · What each box is measured against](#) ← spine 9

[17 · What creates a stacking context](#) ← spine 9

[20 · overflow](#) ← spine 9

[23 · Flexbox · on the container](#) ← spine 5

[26 · Which axis is which](#) ← spine 16

[28 · Flexbox · on an item](#) ← spine 16

[31 · The flex shorthand](#) ← spine 16

[35 · @media \(min-width: 48em\) — the one you are allowed](#) ← spine 25

[36 · The five that will cost you time](#) ← spine 16

[37 · What float is still for](#) ← spine 22

[38 · Where to look things up](#) ← spine 29